```
55
56          // 处理列
57          sort(col, col + m, cmp);
58          // 选 k - cnt 列
59          for(int i = 0; i < k - cnt; i++) sum += col[i];
60          ret = max(ret, sum);
61      }
62
63      cout << ret << endl;
64
65      return 0;
66  }
```

8.2 推公式

如果细说的话，这个专题应该叫**推公式+排序**。其中推公式就是**寻找排序规则**，排序就是在该排序规则下**对整个对象排序**。

在解决某些问题的时，当我们发现**最终结果需要调整每个对象的先后顺序**，也就是对整个对象排序时，那么我们就可以用推公式的方式，得出我们的排序规则，进而对整个对象排序。

正确性证明：

> 利用排序解决问题，最重要的就是需要证明"**在新的排序规则下，整个集合可以排序**"。这需要用到**离散数学中"全序关系"的知识**。我会在第一道题中证明该题的排序规则下，整个集合是可以排序的。
>
> 但是证明过程很麻烦，后续题目中我们**只要发现该题最终结果需要排序，并且交换相邻两个元素的时候，对其余元素不会产生影响**，那么我们就可以推导出排序的规则，然后直接去排序，就不去证明了。

8.2.1 拼数

> **题目来源：** 洛谷
>
> **题目链接：** P1012 [NOIP1998 提高组] 拼数
>
> **难度系数：** ★★

【题目描述】

> 设有 n 个正整数 $a_1, a_2, ..., a_n$ ，将它们联接成一排，相邻数字首尾相接，组成一个最大的整数。

【输入描述】

> 第一行有一个整数，表示数字个数 n 。
>
> 第二行有 n 个整数，表示给出的 n 个整数 a_i 。

> 对于全部的测试点，保证 $1 \leq n \leq 20$，$1 \leq a_i \leq 10^9$

【输出描述】

> 一个正整数，表示最大的整数

【示例一】

> 输入：
>
> 3
>
> 13 312 343
>
> 输出：
>
> 34331213

【解法】

我们发现，任取序列里面相邻的两项 $a[i], a[i+1]$，交换他们的顺序，并不影响 $[1, i-1]$ 与 $[i+1, n]$ 之间每一位的权值。因此我们可以找一种比较方式，对整个数组排序，最终的结果就是最优序列。

设两个相邻的数对应的字符串形式为 x, y，因为要的是最大值，所以自定义比较方式：

- $x+y > y+x$： x 放在前面， y 放在后面；
- $x+y < y+x$： y 放在前面， x 放在后面；
- $x+y = y+x$：谁前谁后无所谓。

【参考代码】

代码块

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 25;

int n;
string a[N];

bool cmp(string& x, string& y)
{
    return x + y > y + x;
```

```cpp
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    // 排序
    sort(a + 1, a + 1 + n, cmp);

    for(int i = 1; i <= n; i++) cout << a[i];

    return 0;
}
```

8.2.2 保卫花园

> **题目来源：** 洛谷
>
> **题目链接：** [USACO07JAN] Protecting the Flowers S
>
> **难度系数：** ★★★

【题目描述】

> 有 n 头奶牛跑到 FJ 的花园里去吃花儿了，它们分别在距离牛圈 T_i （这里指 FJ 到那里需要 T_i 分钟) 处吃花，每分钟会吃掉 D_i 朵花，FJ 现在要将它们给弄回牛圈，但是他每次只能弄一头回去，来回用时总共为 $2 \times T_i$ 分钟，在这段时间内，其它的奶牛会继续吃 FJ 的花，速度保持不变，当然正在被赶回牛圈的奶牛不能继续吃了。现在求在最好的方案下奶牛吃掉花的最小朵数。

【输入描述】

> 第一行：单个整数 n
>
> 后面 n 行：每行包含两个空格分隔的整数 T_i 和 D_i ，它们描述了一头奶牛的特征

【输出描述】

> 单独一行: 单个整数，表示被破坏的花的最小数量

【示例一】

> 输入：
>
> 6
>
> 3 1
>
> 2 5
>
> 2 3

```
3 2
4 1
1 6
```

输出：

```
86
```

【解法】

我们发现，在在一个序列中，任意交换相邻两头牛 $a[i], a[i+1]$ 的顺序之后，区间 $[1, i-1]$ 以及 $[i+1, n]$ 内所有牛吃草的总量不变。因此我们可以找一种比较方式，对整个数组排序，最终的结果就是最优序列。

设交换的相邻两头牛的下标为 i, j $(i+1=j)$，到达 i 位置时，经过的时间为 T。由此可得：

- 交换前：

  第 i 头牛吃草量： $T \times d_i$；

  第 j 头牛的吃草量： $(T + 2 \times t_i) \times d_j$

  消耗的总时长： $sum_1 = Td_i + Td_j + 2t_id_j$

- 交换后：

  第 j 头牛吃草量： $T \times d_j$；

  第 i 头牛的吃草量： $(T + 2 \times t_j) \times d_i$

  消耗的总时长： $sum_2 = Td_j + Td_i + 2t_jd_i$

如果 i 在 j 前比较好，对应的 sum_1 应该小于 sum_2，也就是 $t_id_j < t_jd_i$。

综上，我们自定义的比较方式为：

- 当 $t_id_j < t_jd_i$ 时： i 在 j 前；
- 当 $t_id_j > t_jd_i$ 时： i 在 j 后；
- 当 $t_id_j = t_jd_i$ 时：谁前谁后无所谓。

【参考代码】

代码块

```cpp
#include <iostream>
#include <algorithm>

using namespace std;
```

```cpp

typedef long long LL;

const int N = 1e5 + 10;

int n;
struct node
{
    int t;
    int d;
}a[N];

bool cmp(node& x, node& y)
{
    return x.t * y.d < y.t * x.d;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i].t >> a[i].d;
    }

    sort(a + 1, a + 1 + n, cmp);

    LL ret = 0, t = 0;
    for(int i = 1; i <= n; i++)
    {
        ret += a[i].d * t;
        t += 2 * a[i].t;
    }

    cout << ret << endl;

    return 0;
}
```

8.2.3 奶牛玩杂技

> **题目来源：** 洛谷
>
> **题目链接：** [USACO05NOV] 奶牛玩杂技

> **难度系数：** ★★★

【题目描述】

> Farmer John 养了 N 头牛，她们已经按 $1 \sim N$ 依次编上了号。FJ 所不知道的是，他的所有牛都梦想着从农场逃走，去参加马戏团的演出。可奶牛们很快发现她们那笨拙的蹄子根本无法在钢丝或晃动的的秋千上站稳（她们还尝试过把自己装在大炮里发射出去，但可想而知，结果是悲惨的）。最终，她们决定练习一种最简单的杂技：把所有牛都摞在一起， 比如说， 第一头牛站在第二头的身上， 同时第二头牛又站在第三头牛的身上...最底下的是第 N 头牛。
>
> 每头牛都有自己的体重以及力量，编号为 i 的奶牛的体重为 W_i ，力量为 S_i 。
>
> 当某头牛身上站着另一些牛时它就会在一定程度上被压扁，我们不妨把它被压扁的程度叫做它的压扁指数。对于任意的牛，她的压扁指数等于摞在她上面的所有奶牛的总重（当然不包括她自己）减去它的力量。奶牛们按照一定的顺序摞在一起后， 她们的总压扁指数就是被压得最扁的那头奶牛的压扁指数。
>
> 你的任务就是帮助奶牛们找出一个摞在一起的顺序，使得总压扁指数最小。

【输入描述】

> 第一行一个整数 N 。
>
> 接下来 N 行，每行两个整数 W_i 和 S_i 。
>
> 对于 100% 的数据， $1 \leq N \leq 5 \times 10^4, 1 \leq W_i \leq 10^4, 1 \leq S_i \leq 10^9$ 。

【输出描述】

> 一行一个整数表示最小总压扁指数。

【示例一】

> 输入：
>
> 3
>
> 10 3
>
> 2 5
>
> 3 3
>
> 输出：
>
> 2

【解法】

我们发现，在在一个序列中，任意交换相邻两头牛 $a[i], a[i+1]$ 的顺序之后，区间 $[1, i-1]$ 以及 $[i+1, n]$ 内每一头牛的压扁指数都是不变。因此我们可以找一种比较方式，对整个数组排序，最终的结果就是最优序列。

设交换的相邻两头牛的下标为 i, j $(i+1=j)$，$[1, i-1]$ 区间内所有奶牛的重量为 W。由此可得：

- 交换前：

  第 i 头牛的压扁指数为：$W - s_i$；

  第 j 头牛的压扁指数为：$W + w_i - s_j$；

  被压得最扁的那头奶牛的压扁指数：$k_1 = max(W - s_i,\ W + w_i - s_j)$

- 交换后：

  第 j 头牛的压扁指数为：$W - s_j$；

  第 i 头牛的压扁指数为：$W + w_j - s_i$；

  被压得最扁的那头奶牛的压扁指数：$k_2 = max(W - s_j,\ W + w_j - s_i)$

由于 $k1, k2$ 里面都包含 W，可以统一删除。如果 i 在比 j 在前比较好，也就是总体的压扁指数要尽可能的小。那么得要求 $k1 < k2$，也就是：$max(-s_i, w_i - s_j) < max(-s_j, w_j - s_i)$。

【参考代码】

代码块

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

typedef long long LL;

const int N = 5e4 + 10;

int n;
struct node
{
    int w, s;
}a[N];

bool cmp(node& i, node& j)
{
    return i.w + i.s < j.w + j.s;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i].w >> a[i].s;
```

```
25
26      sort(a + 1, a + 1 + n, cmp);
27
28      LL ret = -1e9 - 10, w = 0;
29      for(int i = 1; i <= n; i++)
30      {
31          ret = max(ret, w - a[i].s);
32          w += a[i].w;
33      }
34      cout << ret << endl;
35
36      return 0;
37  }
```

8.3 哈夫曼编码

> 主要记一下流程。

8.3.1 哈夫曼编码

> **题目来源：** 牛客网
>
> **题目链接：** 【模板】哈夫曼编码
>
> **难度系数：** ★★

【题目描述】

> 给出一个有 n 种字符组成的字符串，其中第 i 种字符出现的次数为 a_i。请你对该字符串应用哈夫曼编码，使得该字符串的长度尽可能短，求编码后的字符串的最短长度。

【输入描述】

> 第一行输入一个整数 $n(1 \le n \le 2 \times 10^5)$，表示字符种数。
>
> 第二行输入 n 个整数 $a_i(1 \le a_i \le 10^9)$，表示每种字符的出现次数。

【输出描述】

> 输出一行一个整数，表示编码后字符串的最短长度。

【示例一】

> 输入：
>
> 3
>
> 1 2 3